Recursion and the Problem of Operator Associativity

The following slides address a very practical problem that you will encounter when implementing your parser in the project

Recap: Eliminating Left Recursion

- We have seen that LL(k) parsers cannot handle grammars with left recursion
- Solution seen in this course: Eliminate left recursion by changing the grammar (without changing the language)
- Example

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid a \mid b$$

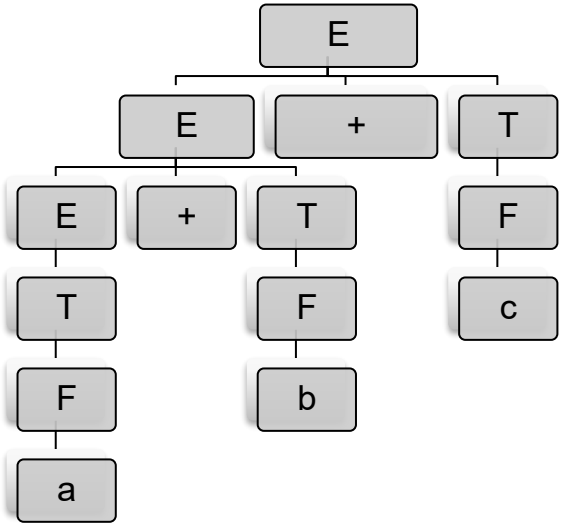
can be transformed to

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid a \mid b$$

Syntax Tree for Transformed Grammar

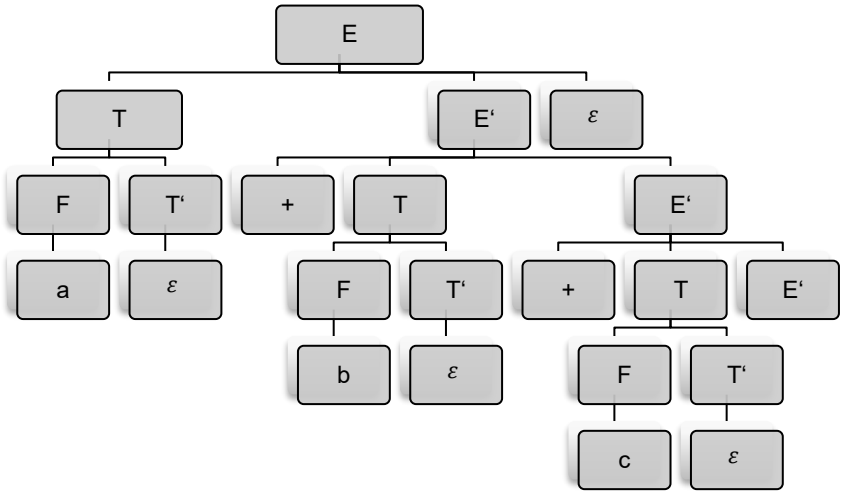
- The transformation of the grammar preserves the generated language but *not the syntax tree*
- Example: $\boldsymbol{a + b + c}$
- Syntax tree with left-recursive grammar

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid a \mid b \mid c$$



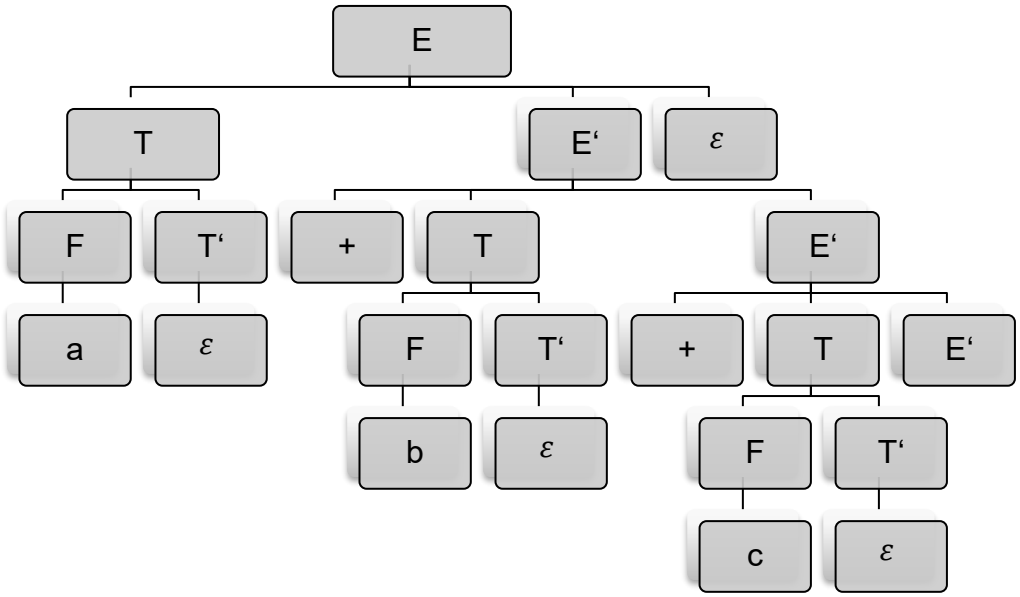
- Syntax tree with transformed grammar

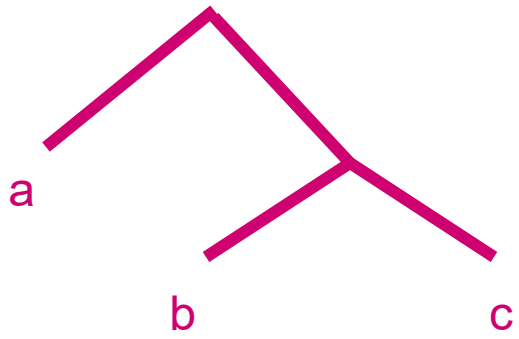
$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow * FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid a \mid b \mid c$$



Problem of Operator Associativity

- Not so nice: The shape of the syntax tree of the transformed grammar seems to suggest that the + operator is right-associative, i.e. the tree of $a + b + c$ now looks like $a + (b + c)$





- That looks strange... In most programming languages the + operator is not right associative. It is a consequence of the elimination of the left recursion. We would like a syntax tree that is closer to the semantic of our programming language.

Possible Solutions

- Many solutions exist to represent left associativity in the tree:
  1. **Use a parser that doesn't have problems with recursive grammars, for example an *LR*-parser (we will see them soon)**
     - LR-parsers are more difficult to implement by hand than LL-parsers. People use parser generator tools for that.
  2. **Use a top-down parser with right-to-left parsing instead of $LL(k)$ left-to-right parsing**
     - This works for this case, but then we will have the same problem for operators for which we *want* right associativity
  3. **After finishing parsing, transform the syntax tree from right to left associativity**
     - Not difficult, just more code to write. And parsing becomes slower.
  4. **"Repair" the syntax tree during parsing**
     - See the next slide →

Repair the syntax tree during parsing

- Let's look at these three rules:

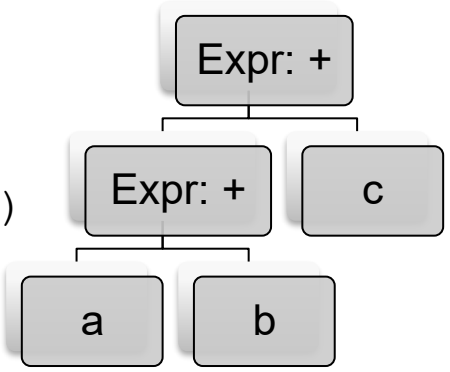
$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$

- If we use the *Kleene-star* notation, we can see better what is happening:

$$E \rightarrow T\ (+T)^*$$

- We can implement the Kleene-star as a while-loop in the parser. This hand-written parser builds a nice AST for E:

```
function parseExpr() {
    term = parseTerm()
    while(lookahead==Plus) {
        term = new Expr(Plus, term, parseTerm())
    }
    return term
}
```



Repair the syntax tree during parsing (2)

- We can do the same for the other rules
- We write the rules

$$T \rightarrow FT'$$
$$T' \rightarrow * FT' \mid \varepsilon$$

  using the *Kleene-star* notation as:

$$T \rightarrow F(* F)^*$$

- and we can see how to write the parser for T with a while-loop:

```
function parseTerm() {
    fact = parseFactor()
    while(lookahead==Multiply) {
        fact = new Factor(Multiply, fact, parseFactor())
    }
    return factor
}
```

Conclusion

- In a hand-written recursive-descent parser, you can parse left-associative operators with a loop.

  (Right-associative operators can be still parsed with right recursion)

- This shows the advantage of a hand-written recursive descent parser: You have full control over the parsing process and can generate the AST in the way you want it

Drawback of our solution

- You need a loop for *each* precedence level in the language
  - In our example: One loop for E, one loop for T
- In languages with many precedence levels (C has 15 levels!!!), the parsing becomes slow. To parse a simple expression like

  myVariable

  the code goes at least once through the head of the while-loops of all precedence levels:

```
parseExpr:
   term = parseTerm()          →   parseTerm:
                                      fact = parseFactor()
                                      while(lookahead==Multiply)...

   while(lookahead==Plus) ...
```

- The speed can be improved with an algorithm called “precedence climbing” (which is used in the Java compiler!)
  https://www.engr.mun.ca/~theo/Misc/exp_parsing.htm
- Not a problem for the small language in our project ☺